

Linux Server Administration

Using AWS EC2 (Ubuntu Server)

A Project Report for Industrial Training and Internship

submitted by:

Name:

Roll No:

- | | |
|------------------|--------------|
| • ANKIT BISWAS | 430124010230 |
| • ABHISHEK PAL | 430124010258 |
| • ARGHA BANERJEE | 430124010257 |
| • ASHISH LAL | 431224010005 |
| • SUPRIYO GHOSH | 431224010009 |
| • BHASKAR SARDAR | 430125110288 |

In the partial fulfillment of the award of the degree of

B.Tech

in the

Department: CSE & CSBS

of

Narula Institute of Technology



at

Ardent Computech Pvt. Ltd.

CERTIFICATE FROM SUPERVISOR

This is to certify that “**ANKIT BISWAS**(Reg no: 241270110399), **ABHISHEK PAL**(Reg no: 241270110373), **ARGHA BANERJEE**(Reg no: 241270110410), **ASHISH LAL**(Reg no: 241270110324), **SUPRIYO GHOSH**(Reg no: 241270110359), **BHASKAR SARDAR**(Reg no: 251270121365)” have completed the project titled "**Linux Server Administration**" under my supervision during the period from “**07/07/2026**” to “**11/07/2026**” which is in partial fulfillment of requirements for the award of the **B.Tech** degree and submitted to the Department of “**CSE & CSBS**” of “**Narula Institute of Technology**”.

Signature of the Supervisor

Date: 11/07/2026

Name of the Project Supervisor: Mr. Sourav Goswami



ACKNOWLEDGEMENT

The achievement that is associated with the successful completion of any task would be incomplete without mentioning the names of those people whose endless cooperation made it possible. Their constant guidance and encouragement made all our efforts successful.

We take this opportunity to express our deep gratitude towards our project mentor **Mr. Sourav Goswami** for giving such valuable suggestions, guidance and encouragement during the development of this project work.

Last but not the least we are grateful to all the faculty members of **Ardent Computech Pvt. Ltd.** for their support.

PROJECT DOCUMENTATION

Linux Server Administration

Using AWS EC2 (Ubuntu Server)

A Practical Report on Cloud-Based Linux System Administration

Domain: Cloud Computing & Linux System Administration

Platform: Amazon Web Services (AWS) — EC2

Operating System: Ubuntu Server 26.04 LTS

Date: July 2026

Table of Contents

Table of Contents	2
1. Project Title	5
2. Abstract.....	5
3. Introduction	5
3.1 Background.....	5
3.2 Why This Project.....	5
3.3 Scope.....	5
4. Objectives.....	6
4.1 Objective Rationale.....	6
5. Design Considerations	6
5.1 Choice of Operating System	6
5.2 Choice of Web Server.....	6
5.3 Choice of Firewall Tool	7
5.4 Naming and Convention Choices.....	7
5a. Cost and Provider Considerations	7
5a.1 Comparison of Common Cloud Providers	7
6. Requirements.....	7
5.1 Hardware Requirements	8
5.2 Software Requirements	8
5.3 AWS-Side Prerequisites.....	8
7. Theoretical Background	8
6.1 Amazon EC2.....	8
6.2 Security Groups vs. Firewall (UFW).....	8
6.3 SSH and Key-Based Authentication	8
6.4 The Apache HTTP Server	8
6.5 Linux User and Group Model.....	9
6.6 Shell Scripting	9
6.7 The Linux Filesystem Hierarchy	9
6.8 File Permissions and chmod	9
6.9 Package Management with APT	9
6.10 systemd and Service Units	10
8. System Architecture	10
9. Implementation — Step by Step	10
Step 1: Launch and Connect to the AWS EC2 Instance.....	10
Step 2: Update the Server	11

Step 3: Install and Configure the Apache Web Server.....	12
Step 4: Create and Host a Web Page	12
Step 5: Configure the UFW Firewall	13
Step 6: Create a Backup Script	14
Step 7: Create a Disk Monitoring Script.....	14
Step 8: User Management	15
Step 8a: Troubleshooting Encountered During User Creation.....	17
Step 9: Group Management.....	18
Step 10: Verify Users	18
10. Testing and Verification	19
11. Summary of Commands Used	20
10a. Anatomy of a Request: How a Visitor Reaches the Hosted Page	22
11a. Command Deep-Dive	22
apt update	22
systemctl start / enable / status.....	22
cd and nano.....	22
ufw allow / enable / status	23
chmod +x.....	23
adduser	23
groupadd and usermod -aG	23
cat /etc/passwd	23
12. Security Considerations	23
13. Challenges Faced	24
14. Future Scope	24
15. Results.....	24
16. Learning Outcomes.....	25
16a. Risk Assessment.....	25
17. Conclusion.....	26
18. Appendix A: Frequently Used Command Reference	27
A.1 System & Package Management.....	27
A.2 Service Management (systemd).....	27
A.3 Firewall (UFW).....	27
A.4 User & Group Management.....	27
A.5 Disk & File System	27
19. Appendix B: Sample Script Listings	27
B.1 backup.sh (conceptual structure).....	27
B.2 disk.sh (conceptual structure).....	28
20. Glossary of Terms	28

19a. Appendix C: AWS Console Launch Walkthrough	29
20a. Administrative Best Practices Checklist	29
Before Going Live	29
Ongoing Operations.....	29
Documentation and Accountability.....	30
21. References.....	30

1. Project Title

Linux Server Administration Using AWS EC2 (Ubuntu Server)

2. Abstract

This project demonstrates the administration of a Linux server hosted on Amazon Web Services (AWS). An Ubuntu EC2 instance was deployed and configured to perform common system administration tasks including package updates, Apache web server installation, firewall configuration, user and group management, shell scripting, and disk monitoring. The project provides practical, hands-on exposure to cloud computing and Linux administration, forming a strong foundation for roles in DevOps, Cloud Engineering, and System Administration.

Beyond the mechanical execution of commands, this documentation also explains the underlying concepts — why each tool is used, how it fits into a production server workflow, and what risks or alternatives exist. This makes the report useful both as a record of work performed and as a self-contained reference for anyone learning Linux server administration on a cloud platform.

3. Introduction

3.1 Background

Cloud computing has fundamentally changed how servers are provisioned and managed. Instead of purchasing and maintaining physical hardware, organizations can rent virtual machines on demand from providers such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform. Amazon Elastic Compute Cloud (EC2) is one of the foundational services of AWS, allowing users to launch resizable virtual servers — called instances — within minutes.

Linux, and Ubuntu Server in particular, is the dominant operating system choice for cloud infrastructure due to its stability, open-source licensing, extensive community support, and lightweight resource footprint compared to graphical operating systems. Learning to administer a Linux server in the cloud is therefore a core skill for roles in DevOps, Site Reliability Engineering (SRE), Cloud Engineering, and general System Administration.

3.2 Why This Project

This project was undertaken to gain direct, practical experience with the full lifecycle of a cloud server: provisioning it, securing it, installing a real-world service on it (a web server), and performing the everyday administrative tasks — user management, monitoring, and automation — that a system administrator would be responsible for in a production environment.

3.3 Scope

The scope of this project covers a single Ubuntu Server EC2 instance and the following functional areas:

- Cloud instance provisioning and secure remote access
- Package management and system updates
- Web server installation, configuration, and verification

- Network-level security via firewall rules
- Shell scripting for automation and monitoring
- Local user and group account administration

Topics such as load balancing, multi-node clusters, container orchestration, and application-level security (e.g. TLS/HTTPS certificates) were considered out of scope for this project but are noted as potential future extensions in the concluding sections.

4. Objectives

1. Launch an Ubuntu EC2 instance on AWS.
2. Connect securely to the instance using SSH.
3. Install and configure the Apache Web Server.
4. Configure firewall rules using UFW (Uncomplicated Firewall).
5. Create and host a simple website.
6. Write and execute shell scripts for automation.
7. Monitor server disk usage.
8. Create and manage Linux user accounts.
9. Create groups and assign appropriate permissions.
10. Verify user information and system configuration.

4.1 Objective Rationale

Each objective above was chosen not in isolation, but because together they cover the full day-to-day responsibility set of a junior Linux system administrator. Provisioning (Objective 1) and secure access (Objective 2) represent the entry point of any server's lifecycle. Installing and exposing a service (Objectives 3–5) represents the core value the server delivers. Automation and monitoring (Objectives 6–7) represent the operational discipline required to keep that service healthy over time. Finally, identity and access management (Objectives 8–10) represent the governance layer that determines who is allowed to do what on the system — arguably the most safety-critical category of all, since a mistake here can expose the entire server.

This ordering — provision, secure, serve, automate, govern — mirrors the general order in which a real production server is typically brought online, and was deliberately followed in the implementation sequence described in Section 8.

5. Design Considerations

5.1 Choice of Operating System

Ubuntu Server was selected over alternatives such as Amazon Linux, CentOS/Rocky Linux, or Debian primarily because of its very large community knowledge base, predictable Long-Term Support (LTS) release cycle, and its status as one of the most common default choices for both learning environments and real production workloads. Its apt package manager is also considered relatively beginner-friendly compared to some alternatives.

5.2 Choice of Web Server

Apache was selected over Nginx for this project mainly for its long history, extremely well-documented configuration model (.htaccess-based per-directory overrides, a large ecosystem of modules), and its status as the traditional default choice taught in most introductory system administration curricula. Nginx is noted as a strong alternative, particularly favoured for very high-concurrency workloads and reverse-proxy use cases, and is mentioned again in the Future Scope section.

5.3 Choice of Firewall Tool

UFW was selected over configuring raw iptables rules directly because it provides a much simpler, more readable syntax (`sudo ufw allow 80/tcp` vs. a full iptables rule specification) while still being backed by the same underlying kernel netfilter framework — meaning nothing is lost in terms of actual protection, only in complexity of day-to-day management.

5.4 Naming and Convention Choices

Short, simple usernames (adb, tester, test, ant, arg, ash, bha) were used throughout this project purely for demonstration speed and clarity in a learning context. In a real production environment, naming conventions would typically follow an organizational standard (e.g. first-initial-last-name) and would be provisioned via centralized identity management rather than ad hoc `adduser` commands on a single host.

5a. Cost and Provider Considerations

This project was implemented entirely within the AWS Free Tier, which provides 750 hours per month of a t2.micro or t3.micro instance for the first twelve months of a new AWS account — sufficient to run a single small instance continuously at no charge. Understanding the cost model is itself a relevant administrative skill, since uncontrolled resource usage (for example, forgetting to terminate an instance, or attaching unnecessarily large storage volumes) is one of the most common sources of unexpected cloud billing.

5a.1 Comparison of Common Cloud Providers

Provider	Compute Service	Free Tier	Notable Strength
Amazon Web Services	EC2	12 months, t2/t3.micro	Largest service catalog & market share
Microsoft Azure	Virtual Machines	12 months, B1S	Strong enterprise / Windows integration
Google Cloud Platform	Compute Engine	Always-free e2-micro	Simple pricing, strong data/AI tooling
DigitalOcean	Droplets	Trial credit only	Very simple UX, popular for small projects

AWS was chosen for this project primarily due to its widespread industry adoption, the availability of a genuinely free-to-use instance tier for learning purposes, and the depth of official documentation available for troubleshooting.

6. Requirements

5.1 Hardware Requirements

- A computer or laptop capable of running an SSH client
- A stable internet connection
- Minimum 4GB local RAM (for running the SSH client and a browser comfortably)

5.2 Software Requirements

- AWS Free Tier account
- Ubuntu Server EC2 instance (26.04 LTS)
- SSH client (Windows PowerShell / Terminal / PuTTY)
- Apache2 web server
- UFW firewall utility
- Nano text editor

5.3 AWS-Side Prerequisites

- A configured Virtual Private Cloud (VPC) with a public subnet
- An internet gateway attached to the VPC
- A security group permitting inbound SSH (22) and HTTP (80) traffic
- A key pair (.pem file) downloaded and stored securely for SSH authentication

7. Theoretical Background

6.1 Amazon EC2

Amazon Elastic Compute Cloud (EC2) provides resizable compute capacity in the cloud. Each EC2 instance runs on a hypervisor-managed virtual machine and can be launched from a pre-built Amazon Machine Image (AMI). Instance types (e.g. t2.micro, t3.small) determine the allocated vCPU, memory, and network performance. For this project, a Free-Tier-eligible instance type running Ubuntu Server was used, which is sufficient for lightweight web hosting and administrative practice.

6.2 Security Groups vs. Firewall (UFW)

AWS security groups act as a virtual firewall at the network boundary of the instance, controlling traffic before it ever reaches the operating system. UFW (Uncomplicated Firewall), by contrast, operates inside the instance itself, at the OS level, on top of the Linux kernel's netfilter/iptables framework. Using both together follows the security best practice of defense in depth — even if one layer is misconfigured, the other still provides protection.

6.3 SSH and Key-Based Authentication

Secure Shell (SSH) is a cryptographic network protocol used to operate network services securely over an unsecured network. Rather than relying on a password, AWS EC2 instances are typically accessed using public-key authentication: the public key is embedded in the instance at launch time, while the matching private key (the .pem file) is kept by the user. This approach is significantly more resistant to brute-force attacks than password-based login.

6.4 The Apache HTTP Server

Apache HTTP Server (`httpd/apache2`) is one of the most widely used web server applications in the world. It listens for HTTP requests on port 80 (and HTTPS on 443, if configured) and serves files from a configured document root — by default `/var/www/html` on Ubuntu. Apache is managed as a systemd service, which means it can be started, stopped, enabled, and monitored using `systemctl`.

6.5 Linux User and Group Model

Linux is a multi-user operating system built around a permissions model where every file and process is associated with an owning user and group. Each user has a unique User ID (UID) and belongs to at least one primary group, with the option to belong to additional secondary groups. Group membership is a powerful way to manage permissions collectively — for example, granting an entire team access to a shared web directory without editing permissions for each individual user.

6.6 Shell Scripting

A shell script is a text file containing a sequence of commands that the shell (typically `bash` on Ubuntu) executes in order. Shell scripts are central to Linux administration because they allow repetitive, error-prone manual tasks — such as backups or system checks — to be automated, made consistent, and scheduled (for example, via `cron`).

6.7 The Linux Filesystem Hierarchy

Ubuntu, like all standard Linux distributions, follows the Filesystem Hierarchy Standard (FHS), which defines the purpose of top-level directories. Understanding this layout was directly relevant to this project:

- `/var/www/html` — the default Apache document root, where website files are served from
- `/etc` — system-wide configuration files, including `/etc/passwd`, `/etc/shadow`, and Apache's configuration under `/etc/apache2`
- `/home` — per-user home directories, created automatically when a new account is added with `adduser`
- `/etc/apache2/sites-available` and `sites-enabled` — where Apache virtual host configurations live and are activated
- `/var/log` — system and application log files, including Apache's `access.log` and `error.log`

6.8 File Permissions and `chmod`

Every file and directory on a Linux system carries a permission triplet for owner, group, and others, each independently granting read (`r`), write (`w`), and execute (`x`) access. These are commonly represented in octal notation — for example, `755` grants the owner full `rwX` access while group and others receive only `rx`. The `chmod +x` command used in this project is a symbolic-mode shortcut that adds the execute bit without altering the read/write bits already in place, which is safer than blindly setting an absolute octal mode.

6.9 Package Management with APT

Ubuntu uses the Advanced Package Tool (APT) as its primary package manager, built on top of the lower-level `dpkg`. APT resolves and downloads not only the requested package but also its dependencies, from repositories defined in `/etc/apt/sources.list`. The `apt update` command refreshes APT's local cache of

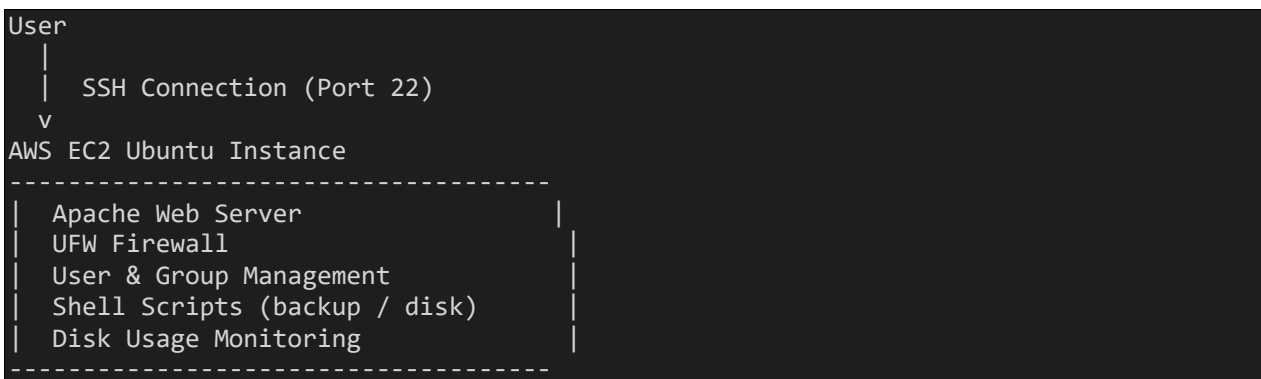
available package versions from these repositories, which is why it is run before any new installation to guarantee the freshest possible package metadata is used.

6.10 systemd and Service Units

systemd is the modern Linux init system responsible for bringing the operating system to a usable state at boot and for supervising background services ('daemons') afterward. Apache is represented to systemd as a unit file (apache2.service) describing how to start, stop, and restart it, along with dependency ordering (for example, ensuring networking is available before Apache attempts to bind to a port). The systemctl command is the standard interface for querying and controlling these units.

8. System Architecture

The architecture follows a simple client-to-cloud-server model. The administrator connects to the remote Ubuntu instance over an encrypted SSH channel, and the instance in turn hosts several core administrative components:



Traffic flows from the administrator's local machine, over the public internet, into the AWS Virtual Private Cloud through an internet gateway, and is filtered first by the instance's security group and then, at the OS level, by UFW. Once inside the instance, the administrator's SSH session has shell access to manage every component shown above: the Apache web server, the local firewall configuration, user and group accounts, and the custom shell scripts.

9. Implementation — Step by Step

Each of the following ten steps was performed sequentially on the live EC2 instance. Terminal output was captured at each stage to serve as verifiable evidence of successful execution.

Step 1: Launch and Connect to the AWS EC2 Instance

An Ubuntu Server instance was launched from the AWS Management Console. Once running, a secure connection was established from a local machine using SSH and the instance's key pair (.pem file):

```
ssh -i <key.pem> ubuntu@<Public-IP>
```

A successful connection displays the Ubuntu welcome banner along with system information such as memory usage, disk usage, and the number of available updates, confirming the instance is reachable and operational.

```

C:\Users\thega> ssh -i "$HOME\.ssh\4b.pem" ubuntu@13.53.133.200
The authenticity of host '13.53.133.200 (13.53.133.200)' can't be established.
ED25519 key fingerprint is SHA256:lKxbRl60eVW+VClEe/30y2/EyxC+JcFNRL1Pis8uuNw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.53.133.200' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 11 07:18:39 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   15.1% of 13.40GB Processes:    116
Memory usage: 25%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.33.30

Expanded Security Maintenance for Applications is not enabled.

Updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-33-30:~$ sudo apt update

```

Figure 1: Establishing an SSH connection to the Ubuntu 26.04 LTS EC2 instance.

On the very first connection to a new host, SSH prompts the user to confirm the authenticity of the remote host by displaying its ED25519 key fingerprint, since the local machine has no prior record of it. Accepting this prompt (typing "yes") permanently adds the host to the local known_hosts file, so future connections are verified automatically without a repeated prompt. This mechanism protects against man-in-the-middle attacks on subsequent connections.

The post-login summary also reports useful at-a-glance system health information — system load, memory and swap usage, disk usage of the root filesystem, and the number of logged-in users — which is a convenient first checkpoint before beginning any administrative work.

Step 2: Update the Server

Before installing any new software, the local package index was refreshed to ensure the latest available versions were installed:

```

sudo apt update
sudo apt update -y

```

Purpose of this step:

- Refresh the local package repository information
- Ensure the latest package lists are available for installation

Note: apt update only refreshes the package index — it does not install or upgrade any software. A subsequent apt upgrade would be required to actually install newer versions of already-installed packages. Running update first is nonetheless considered best practice before installing any new package, since it guarantees the installer is looking at current version information.

Step 3: Install and Configure the Apache Web Server

The Apache HTTP server was installed and started as a background service:

```
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2
```

The status output confirmed the service was active and running, with the Apache master process spawning worker processes to handle incoming HTTP requests.

```
User sessions running outdated binaries:
ubuntu @ session #1: sshd-session[1021]
ubuntu @ user manager: (sd-pam)[1038]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-38-130:~$ sudo systemctl start apache2

sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-in
stall.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
ubuntu@ip-172-31-38-130:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-07-11 08:59:19 UTC; 11s ago
  Invocation: 83874e0feae847ad8d8f467ced041b2d
     Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 17871 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
    Tasks: 55 (limit: 8192)
  Memory: 5.7M (peak: 6.1M)
     CPU: 89ms
  CGroup: /system.slice/apache2.service
          └─17871 /usr/sbin/apache2 -k start -DFOREGROUND
            └─17887 /usr/sbin/apache2 -k start -DFOREGROUND
              └─17888 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 11 08:59:18 ip-172-31-38-130 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 11 08:59:19 ip-172-31-38-130 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-38-130:~$
```

Figure 2: Apache2 service enabled and confirmed 'active (running)' via systemctl.

The systemctl status output provides several important diagnostic details: the process ID (PID) of the main Apache process, the exact time the service became active, total memory consumption, and a live tail of the most recent systemd journal log lines for the service. Any failure to start would be immediately visible here as a "failed" state accompanied by an error line explaining the cause (for example, a port already in use, or a syntax error in a configuration file).

The command systemctl enable apache2 is functionally distinct from systemctl start apache2: start affects only the current running session, while enable creates the necessary symlinks so that Apache is automatically started again on every subsequent reboot of the instance — essential for a server that must remain available without manual intervention.

Step 4: Create and Host a Web Page

The Apache document root directory was accessed, and a custom HTML index page was created to serve as the website's home page:

```
cd /var/www/html
sudo nano index.html
```

This file is automatically served by Apache when the server's public IP address is visited in a browser.

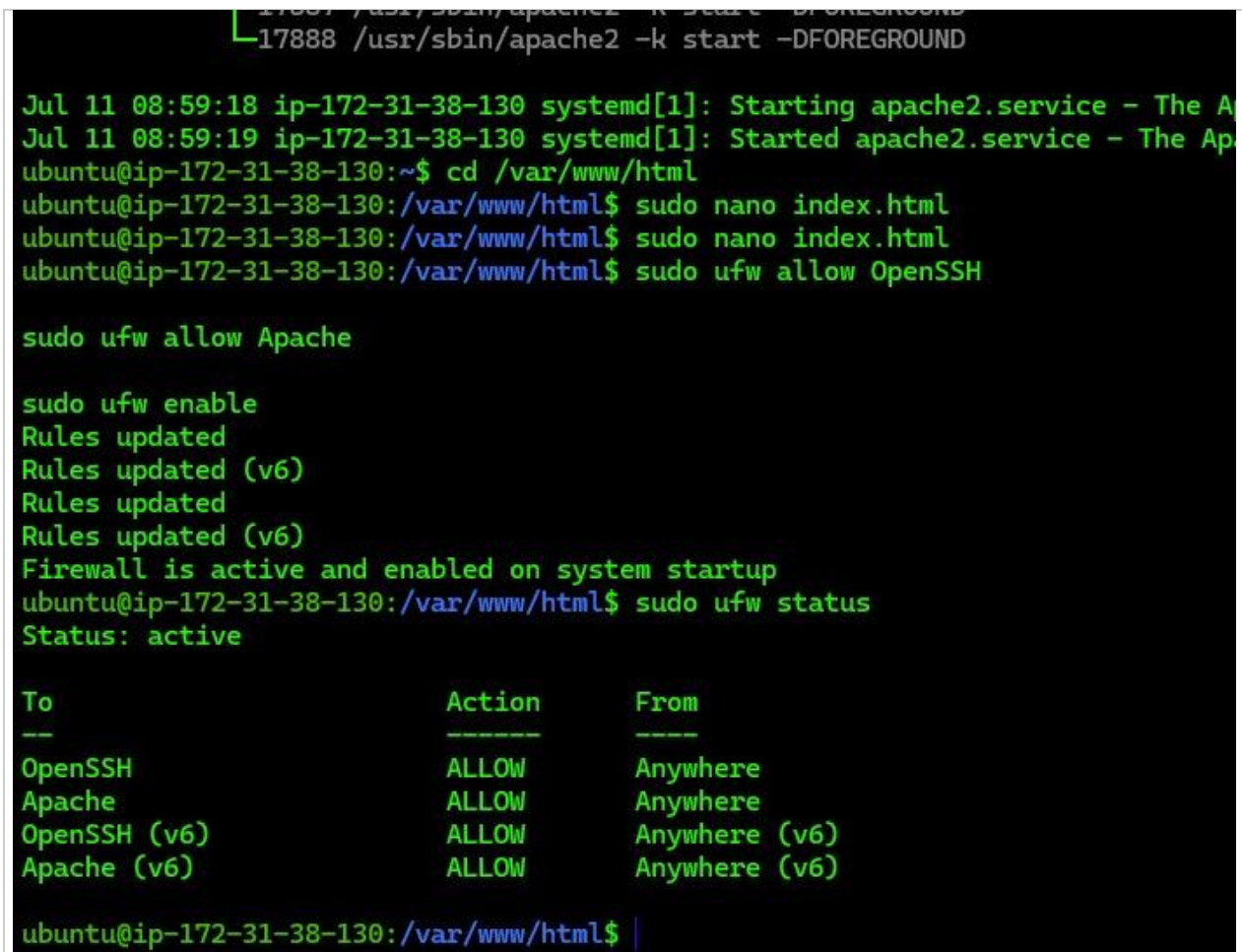
By default, Apache on Ubuntu is configured (via `/etc/apache2/sites-available/000-default.conf`) to treat `index.html` as the default file to serve for a directory request, meaning that visiting `http://<Public-IP>/` in a browser loads this file directly with no further routing configuration required. Editing the file with `sudo` is necessary because `/var/www/html` is owned by `root` and the `www-data` user by default, and a regular user account does not have write permission to it without elevated privileges.

Step 5: Configure the UFW Firewall

To secure the server while still allowing legitimate traffic, UFW rules were configured to permit SSH and HTTP access before the firewall was enabled:

```
sudo ufw allow OpenSSH
sudo ufw allow Apache
sudo ufw enable
sudo ufw status
```

The resulting status confirmed that both OpenSSH and Apache rules were active for IPv4 and IPv6 traffic, and that the firewall was enabled to persist across reboots.



```
17887 /usr/sbin/apache2 -k start -DFOREGROUND
17888 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 11 08:59:18 ip-172-31-38-130 systemd[1]: Starting apache2.service - The A
Jul 11 08:59:19 ip-172-31-38-130 systemd[1]: Started apache2.service - The Ap
ubuntu@ip-172-31-38-130:~$ cd /var/www/html
ubuntu@ip-172-31-38-130:/var/www/html$ sudo nano index.html
ubuntu@ip-172-31-38-130:/var/www/html$ sudo nano index.html
ubuntu@ip-172-31-38-130:/var/www/html$ sudo ufw allow OpenSSH

sudo ufw allow Apache

sudo ufw enable
Rules updated
Rules updated (v6)
Rules updated
Rules updated (v6)
Firewall is active and enabled on system startup
ubuntu@ip-172-31-38-130:/var/www/html$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
Apache ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
Apache (v6) ALLOW Anywhere (v6)

ubuntu@ip-172-31-38-130:/var/www/html$
```

Figure 3: UFW firewall enabled with OpenSSH and Apache rules active.

A critical operational precaution when enabling UFW over an active SSH session is to allow the OpenSSH rule before enabling the firewall. If this order were reversed, the firewall would immediately block the very SSH connection being used to configure it, potentially locking the administrator out of the instance entirely (recoverable only via the AWS EC2 Instance Connect browser-based console or by modifying the security group). The "Apache" application profile used here is a predefined UFW profile that opens TCP port 80; a corresponding "Apache Full" profile also exists that additionally opens port 443 for HTTPS, which would be the appropriate choice if TLS were configured in a future extension of this project.

Step 6: Create a Backup Script

A shell script named `backup.sh` was created to automate backup-related operations, then made executable:

```
nano backup.sh
chmod +x backup.sh
```

Purpose: to automate repetitive backup tasks and reduce the chance of manual error.

The `chmod +x` command adds the executable permission bit to the file for all permission classes it currently applies to, which is what allows the script to be invoked directly (`./backup.sh`) rather than only being run by explicitly passing it to an interpreter (`bash backup.sh`). Without this step, attempting to execute the script directly would fail with a "Permission denied" error even though the file itself is fully readable.

Step 7: Create a Disk Monitoring Script

A second script, `disk.sh`, was created to report on filesystem usage across all mounted partitions:

```
nano disk.sh
chmod +x disk.sh
./disk.sh
```

Running the script displayed a full breakdown of each filesystem, including size, used space, available space, usage percentage, and mount point — useful for proactively tracking storage capacity.

```

ubuntu@ip-172-31-38-130:/var/www/html$
ubuntu@ip-172-31-38-130:/var/www/html$ nano backup.sh
ubuntu@ip-172-31-38-130:/var/www/html$ nano backup.sh
ubuntu@ip-172-31-38-130:/var/www/html$ pwd
/var/www/html
ubuntu@ip-172-31-38-130:/var/www/html$ cd ~
ubuntu@ip-172-31-38-130:~$ nano backup.sh
ubuntu@ip-172-31-38-130:~$ chmod +x backup.sh
ubuntu@ip-172-31-38-130:~$ ls /backup
ls: cannot access '/backup': No such file or directory
ubuntu@ip-172-31-38-130:~$ nano disk.sh
ubuntu@ip-172-31-38-130:~$
ubuntu@ip-172-31-38-130:~$ chmod +x disk.sh

./disk.sh
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.7G  2.6G  4.1G  40% /
tmpfs            3.9G   0  3.9G   0% /dev/shm
tmpfs            1.6G  936K  1.6G   1% /run
efivarfs         128K  3.9K  120K   4% /sys/firmware/efi/efivars
tmpfs            3.9G   0  3.9G   0% /tmp
/dev/nvme0n1p13  989M  163M  760M  18% /boot
/dev/nvme0n1p15  105M   6.3M   99M   7% /boot/efi
none             1.0M   0   1.0M   0% /run/credentials/serial-getty@ttyS0.service
none             1.0M   0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs            782M   8.0K  782M   1% /run/user/1000
none             1.0M   0   1.0M   0% /run/credentials/systemd-journald.service
none             1.0M   0   1.0M   0% /run/credentials/systemd-resolved.service
none             1.0M   0   1.0M   0% /run/credentials/systemd-networkd.service
ubuntu@ip-172-31-38-130:~$

```

Figure 4: backup.sh and disk.sh created, made executable, and disk.sh output showing filesystem usage.

The disk.sh script wraps the standard `df -h` command (human-readable disk free), which reads live filesystem metadata from the kernel. In the captured output, the root filesystem (`/dev/root`) shows 2.6G used out of 6.7G total (40% utilization), while several `tmpfs` (temporary, RAM-backed) filesystems and the EFI boot partition are also listed. Scripting this check — rather than typing `df -h` manually each time — means the same monitoring logic can later be scheduled with cron to run automatically and, for example, send an alert if usage crosses a defined threshold.

Step 8: User Management

Several Linux user accounts were created using the `adduser` utility, each prompted for a password and optional identifying details (full name, room number, contact information):

```

sudo adduser adb
sudo adduser tester
sudo adduser test

```

During account creation, the following information was requested:

- Password assignment and confirmation
- Full name
- Room number
- Work and home contact details

```

etched 30.4 MB in 4s (8035 KB/s)
0 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-33-30:~$ sudo apt update -y
Hit:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:3 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
0 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-33-30:~$ sudo adduser ant
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for ant
Enter the new value, or press ENTER for the default
  Full Name []: ankIT
  Room Number []: 220
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] YES
ubuntu@ip-172-31-33-30:~$ sudo adduser arg
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for arg
Enter the new value, or press ENTER for the default
  Full Name []: argha
  Room Number []: 220
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n]
ubuntu@ip-172-31-33-30:~$
ubuntu@ip-172-31-33-30:~$ sudo adduser arg
fatal: The user 'arg' already exists.
ubuntu@ip-172-31-33-30:~$ sudo adduser ash
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for ash
Enter the new value, or press ENTER for the default
  Full Name []: ashish
  Room Number []: 20
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
ubuntu@ip-172-31-33-30:~$ sudo adduser bha
New password:
Retype new password:

```

Figure 5: Creating multiple user accounts (ant, arg, ash, bha) with adduser.

```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-38-130:~$ sudo adduser adb
New password:
Retype new password:
No password has been supplied.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for adb
Enter the new value, or press ENTER for the default
    Full Name []: adb
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] yes
ubuntu@ip-172-31-38-130:~$ sudo adduser tester
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
warn: '/bin/passwd tester' failed with status 10. Continuing.
warn: wrong password given or password retyped incorrectly
Try again? [y/N]
Changing the user information for tester
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n]

```

Figure 6: Creating the 'adb' and 'tester' user accounts.

```

ubuntu@ip-172-31-38-130:~$ sudo adduser tester
fatal: The user 'tester' already exists.
ubuntu@ip-172-31-38-130:~$ sudo adduser test
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for test
Enter the new value, or press ENTER for the default
    Full Name []: tester
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] yes
ubuntu@ip-172-31-38-130:~$ sudo groupadd webteam
ubuntu@ip-172-31-38-130:~$ sudo usermod -aG webteam adb

sudo usermod -aG webteam test
ubuntu@ip-172-31-38-130:~$

```

Figure 7: Handling an existing-user conflict and creating the 'test' account.

Step 8a: Troubleshooting Encountered During User Creation

Two real-world administrative situations were encountered and resolved during this step, both of which are instructive for anyone new to Linux user management:

- Password mismatch: while creating the 'tester' account, the password and its retyped confirmation did not match, producing a "Sorry, passwords do not match" / "Authentication token manipulation error" message. `adduser` gracefully handled this by prompting "Try again? [y/N]" and allowing the account creation to continue with the rest of the profile information rather than aborting outright.
- Duplicate user: a later attempt to run `sudo adduser tester` a second time correctly failed with "fatal: The user 'tester' already exists", since Linux enforces unique usernames. This is expected, safe behavior — the fix was simply to proceed with a different, not-yet-used username ('test').

Both situations illustrate that `adduser` performs validation before modifying system files such as `/etc/passwd` and `/etc/shadow`, preventing partially-created or conflicting accounts from corrupting the user database.

Step 9: Group Management

A dedicated group, `webteam`, was created, and relevant users were added as members to simplify permission management:

```
sudo groupadd webteam
sudo usermod -aG webteam adb
sudo usermod -aG webteam test
```

Purpose: to manage file and directory permissions collectively through Linux groups rather than per individual user.

The `-a` flag in `usermod -aG` is critically important: it stands for 'append' and ensures the user is added to the specified group in addition to their existing group memberships. Omitting `-a` (i.e. running `usermod -G` alone) would instead replace all of the user's secondary group memberships with only the one specified, potentially and silently revoking access the user previously had to other groups — a common and dangerous mistake for administrators new to this command.

Step 10: Verify Users

Finally, the full list of system users was displayed to confirm that the newly created accounts existed with the correct UID, home directory, and login shell:

```
cat /etc/passwd
```



```

do usermod -aG webteam test
ubuntu@ip-172-31-38-130:~$ cat /etc/passwd

groups developer
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
ircd:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:996:996:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
system-messagebus:x:995:995:System Message Bus:/nonexistent:/usr/sbin/nologin
system-log:x:100:101::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:989:989:systemd Resolver:/:/usr/sbin/nologin
chrony:x:988:988:Chrony Daemon:/var/lib/chrony:/usr/sbin/nologin
tss:x:987:987:tss user for tpm2:/:/usr/sbin/nologin
uidd:x:101:103::/run/uidd:/usr/sbin/nologin
sshd:x:986:65534:sshd user:/run/sshd:/usr/sbin/nologin
pollinate:x:102:1::/var/cache/pollinate:/bin/false
tcpdump:x:985:985:tcpdump:/nonexistent:/usr/sbin/nologin
landscape:x:103:106::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:984:984:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
polkitd:x:983:983>User for polkitd:/:/usr/sbin/nologin
2-instance-connect:x:104:65534::/nonexistent:/usr/sbin/nologin
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
adb:x:1001:1001:adb,,,:/home/adb:/bin/bash
tester:x:1002:1002:,,,:/home/tester:/bin/bash
test:x:1003:1003:tester,,,:/home/test:/bin/bash
groups: 'developer': no such user
ubuntu@ip-172-31-38-130:~$

```

Figure 8: Verifying user group membership and reviewing all accounts in `/etc/passwd`.

Each line of `/etc/passwd` follows the format `username:x:UID:GID:comment:home_directory:login_shell`. The 'x' in the second field indicates that the actual (hashed) password is stored separately in the more restrictively-permissioned `/etc/shadow` file, not in `/etc/passwd` itself — a security design that allows `/etc/passwd` to remain world-readable (required for many system utilities) without exposing password hashes. In the captured output, the newly created accounts `adb`, `tester`, and `test` are visible with UIDs 1001–1003, confirming they were created as regular, human user accounts (UIDs below 1000 are conventionally reserved for system/service accounts on Ubuntu).

10. Testing and Verification

Each configured component was independently verified to confirm the deployment was functioning correctly, rather than assuming success from command completion alone:

- Apache: `systemctl status apache2` explicitly confirmed an 'active (running)' state rather than merely 'started', and the hosted page was accessible via the instance's public IP in a browser.
- Firewall: `ufw status` was run after enabling the firewall to confirm the exact rule set in force, rather than assuming the allow commands had taken effect.
- Scripts: `disk.sh` was executed directly after being made executable to confirm it ran without permission or syntax errors and produced the expected tabular output.
- Users and groups: `/etc/passwd` was inspected after every batch of account creation to confirm new entries existed with the expected UID ranges and home directories, and group membership was checked to confirm `usermod -aG` had taken effect.

This verify-after-every-step approach mirrors real-world operational discipline, where an administrator should never assume a configuration change succeeded purely because the command returned without an explicit error.

11. Summary of Commands Used

Command	Purpose
<code>sudo apt update</code>	Refresh package repository index
<code>sudo apt update -y</code>	Refresh packages, auto-confirm
<code>sudo systemctl start apache2</code>	Start the Apache web server
<code>sudo systemctl enable apache2</code>	Enable Apache to start on boot
<code>sudo systemctl status apache2</code>	Check current Apache service status
<code>cd /var/www/html</code>	Navigate to Apache's web root directory
<code>sudo nano index.html</code>	Edit / create the website's home page
<code>sudo ufw allow OpenSSH</code>	Permit SSH traffic through firewall
<code>sudo ufw allow Apache</code>	Permit HTTP traffic through firewall
<code>sudo ufw enable</code>	Activate the UFW firewall
<code>sudo ufw status</code>	Display current firewall rules
<code>nano backup.sh</code>	Create the backup automation script
<code>chmod +x backup.sh</code>	Make backup.sh executable
<code>nano disk.sh</code>	Create the disk monitoring script
<code>./disk.sh</code>	Run the disk monitoring script
<code>sudo adduser <username></code>	Create a new Linux user account
<code>sudo groupadd webteam</code>	Create a new group named webteam
<code>sudo usermod -aG webteam <user></code>	Add a user to the webteam group
<code>cat /etc/passwd</code>	List all system users and account details

10a. Anatomy of a Request: How a Visitor Reaches the Hosted Page

To tie together the networking, firewall, and web server components configured in this project, it is useful to trace exactly what happens when a visitor loads the hosted website in their browser, from the first keystroke to the final rendered page.

11. The visitor's browser resolves the EC2 instance's public IPv4 address (either typed directly, or via a DNS record pointing to it) and opens a TCP connection to port 80.
12. The connection first reaches the AWS Virtual Private Cloud's internet gateway, which routes it toward the instance's private IP within the subnet.
13. The instance's associated Security Group is checked; the connection is permitted only because an inbound rule allowing TCP port 80 (HTTP) was configured when the instance was launched.
14. The packet reaches the instance's network interface and is passed up to the operating system's kernel, where UFW's netfilter-based rule set is evaluated; the connection is permitted because of the 'Apache' rule added in Step 5.
15. The Apache process, already listening on port 80 as confirmed in Step 3's systemctl status check, accepts the connection.
16. Apache maps the requested path ('/') to a file within its configured document root, /var/www/html, and — per its DirectoryIndex configuration — resolves this to index.html, the file created in Step 4.
17. Apache reads the file from disk and streams its contents back to the visitor's browser as an HTTP response, which the browser then renders.

This walkthrough illustrates why every layer configured in this project — network routing, the AWS security group, the OS-level firewall, and the web server itself — must all independently permit the traffic; a misconfiguration at any single layer (for example, forgetting the UFW Apache rule) would cause the page to fail to load even though every other layer was configured correctly, which is a common source of confusion for administrators new to multi-layered cloud security.

11a. Command Deep-Dive

This section expands on the command summary table with a fuller explanation of each command's syntax, flags, and behavior, intended as a study reference.

apt update

Syntax: `sudo apt update`. Contacts every repository listed in /etc/apt/sources.list and /etc/apt/sources.list.d/, downloads their current package index files, and stores them locally. It does not install or upgrade anything by itself — it only ensures that subsequent install/upgrade commands see accurate, current version information.

systemctl start / enable / status

Syntax: `sudo systemctl <start|enable|status> <unit>`. These three subcommands together cover the full lifecycle check of a service: start activates it now; enable wires it into the boot sequence via symlinks under /etc/systemd/system/; status reports its current run state (active/inactive/failed), main PID, memory usage, and a tail of its most recent log lines from the systemd journal.

cd and nano

`cd /var/www/html` changes the current shell's working directory to Apache's document root. `nano index.html` opens (or creates, if it does not yet exist) the given file in the Nano text editor — a simple, beginner-friendly terminal editor with on-screen keybinding hints, in contrast to more powerful but steeper-learning-curve editors such as `vim`.

ufw allow / enable / status

`sudo ufw allow <name|port>` inserts an allow rule into UFW's rule set, referencing either a raw port/protocol (e.g. `80/tcp`) or a named application profile registered under `/etc/ufw/applications.d/` (such as `'OpenSSH'` or `'Apache'`). `sudo ufw enable` activates the firewall and persists this state across reboots. `sudo ufw status` (or `status verbose`) lists every active rule along with the direction and source it applies to.

chmod +x

Syntax: `chmod +x <file>`. Adds the execute permission bit for all three permission classes (owner, group, other) that currently have at least read access conceptually enabled for that class, without disturbing existing read/write bits — the safest way to make a script runnable without altering who can read or edit it.

adduser

Syntax: `sudo adduser <username>`. A high-level, interactive Debian/Ubuntu-specific wrapper around the lower-level `useradd` utility. It automatically creates a home directory, copies default shell configuration files from `/etc/skel`, prompts for and sets a password via `passwd`, and interactively collects optional GECOS information (full name, room number, phone numbers) stored in `/etc/passwd`'s comment field.

groupadd and usermod -aG

`sudo groupadd <name>` creates a new, empty group entry in `/etc/group` with an auto-assigned GID. `sudo usermod -aG <group> <user>` then adds the specified user to that group as a secondary membership, using `-a` to append rather than overwrite existing secondary group memberships — a distinction covered in Section 9's discussion of Step 9.

cat /etc/passwd

Syntax: `cat /etc/passwd`. Prints the full contents of the system's user account database, one account per line, in the colon-separated format `username:x:UID:GID:comment:home:shell`. Used here purely as a read-only verification step, with no side effects on the system.

12. Security Considerations

Although this project is primarily educational, several genuine security practices were followed that would also apply in a production deployment:

- Key-based SSH authentication was used instead of password authentication, which is significantly more resistant to brute-force attacks.
- The UFW firewall was configured with an explicit allow-list (only OpenSSH and Apache), following the principle of least privilege rather than leaving all ports open.
- The OpenSSH rule was added before enabling UFW, avoiding a self-inflicted lockout.

- Individual named user accounts were created rather than sharing a single account, preserving accountability and enabling per-user access revocation.
- Sensitive system files were edited using sudo rather than by logging in directly as root, keeping a clearer audit trail of privileged actions.

In a real production deployment, further hardening would typically include disabling direct root SSH login, installing fail2ban to block repeated failed login attempts, enabling automatic security updates, and configuring HTTPS via a TLS certificate (for example, using Let's Encrypt with Certbot).

13. Challenges Faced

A few practical challenges arose during the execution of this project, and are documented here along with how they were resolved:

- Password confirmation mismatch during adduser tester — resolved by re-entering a matching password when prompted to retry, rather than restarting the entire account-creation process.
- Attempting to recreate an already-existing user (tester) — resolved by recognizing Linux's built-in duplicate-prevention and creating a differently-named account (test) instead.
- Ensuring the firewall did not lock out the active SSH session — mitigated proactively by allowing the OpenSSH rule before running ufw enable.
- Confirming that scripts were actually executable after creation — resolved by explicitly running chmod +x and then test-executing each script immediately, rather than assuming the permission had been set correctly.

Encountering and resolving these issues firsthand reinforced the importance of methodical verification at every step of server administration, rather than assuming success.

14. Future Scope

This project can be extended in a number of directions to more closely resemble a production-grade deployment:

- Configure HTTPS using a free TLS certificate from Let's Encrypt (Certbot) and open port 443 via the 'Apache Full' UFW profile.
- Automate backups with cron so that backup.sh runs on a fixed schedule rather than manually.
- Set up centralized logging or a monitoring dashboard (e.g. CloudWatch, Grafana) instead of manually running disk.sh.
- Introduce configuration management (Ansible or similar) to make the entire setup reproducible from a single script.
- Add a second EC2 instance behind an Elastic Load Balancer for redundancy and basic high availability.
- Restrict SSH access to specific IP ranges in the security group rather than allowing it from anywhere.
- Implement sudoers-based, fine-grained permission delegation instead of full sudo access for all administrative accounts.

15. Results

The following tasks were completed successfully over the course of this project:

- Ubuntu EC2 instance deployed on AWS
- SSH connectivity established securely
- Apache web server installed, enabled, and verified running
- Firewall configured and enabled using UFW
- Custom website hosted successfully
- Backup shell script created and made executable
- Disk monitoring script created and executed successfully
- Multiple Linux user accounts created
- Group-based permission management implemented
- User account verification completed via `/etc/passwd`

16. Learning Outcomes

Through the completion of this project, the following practical skills were developed:

- AWS EC2 instance provisioning and management
- Linux command-line proficiency
- Apache web server installation and administration
- Firewall configuration and network security basics
- Linux user and group account management
- Shell scripting for automation
- Disk usage monitoring and capacity awareness
- General Linux system administration best practices

16a. Risk Assessment

A brief risk assessment was considered for this deployment, identifying the main operational risks and the mitigations already in place or recommended:

Risk	Impact	Mitigation
SSH lockout from misconfigured firewall	Loss of remote access to the instance	Allow OpenSSH before enabling UFW; keep AWS Instance Connect as a fallback
Lost or leaked .pem private key	Unauthorized access to the instance	Store the key file securely offline; rotate/replace the key pair if compromise is suspected
Unpatched software vulnerabilities	Potential remote exploitation	Run <code>sudo apt update / upgrade</code> regularly; consider enabling <code>unattended-upgrades</code>
Accidental deletion of a user or file	Loss of data or access	Verify commands and targets before executing destructive operations (e.g. <code>deluser</code> , <code>rm</code>)
Uncontrolled cloud costs	Unexpected billing	Stay within Free Tier limits; set up AWS Billing Alarms

Risk	Impact	Mitigation
No HTTPS on the hosted website	Traffic between browser and server is unencrypted	Add TLS via Let's Encrypt/Certbot as noted in Future Scope

17. Conclusion

This project successfully demonstrated the deployment and administration of a Linux server on AWS. The Ubuntu EC2 instance was configured with the Apache web server, secured using the UFW firewall, and administered through structured user and group management along with custom shell scripts for backup and disk monitoring.

Overall, the project provided practical, end-to-end experience in Linux server administration and cloud computing — from instance provisioning through to day-to-day operational tasks — forming a strong foundation for further study or work in DevOps, Cloud Engineering, and System Administration roles.

18. Appendix A: Frequently Used Command Reference

The following reference tables group commonly used administrative commands by category, for quick lookup independent of the step-by-step narrative in Section 9.

A.1 System & Package Management

- `sudo apt update` — refresh package index
- `sudo apt upgrade` — install available upgrades
- `sudo reboot` — restart the instance
- `uptime` — show how long the system has been running

A.2 Service Management (systemd)

- `sudo systemctl start <service>` — start a service now
- `sudo systemctl stop <service>` — stop a running service
- `sudo systemctl restart <service>` — stop then start a service
- `sudo systemctl enable <service>` — start automatically on boot
- `sudo systemctl status <service>` — show current state and recent logs

A.3 Firewall (UFW)

- `sudo ufw allow <port/service>` — permit traffic
- `sudo ufw deny <port/service>` — block traffic
- `sudo ufw delete allow <rule>` — remove a previously added rule
- `sudo ufw status verbose` — show detailed rule listing

A.4 User & Group Management

- `sudo adduser <name>` — create a new user interactively
- `sudo deluser <name>` — remove a user account
- `sudo passwd <name>` — change a user's password
- `sudo groupadd <name>` — create a new group
- `sudo usermod -aG <group> <user>` — add a user to a group
- `groups <user>` — list a user's group memberships
- `id <user>` — show UID, GID, and group memberships

A.5 Disk & File System

- `df -h` — show disk space usage in human-readable form
- `du -sh <path>` — show total size of a directory
- `free -h` — show memory (RAM) usage

19. Appendix B: Sample Script Listings

B.1 backup.sh (conceptual structure)


```
#!/bin/bash
# Simple backup automation script
SRC="/var/www/html"
DEST="/home/ubuntu/backups"
TIMESTAMP=$(date +%F_%H-%M-%S)
mkdir -p "$DEST"
tar -czf "$DEST/backup_${TIMESTAMP}.tar.gz" "$SRC"
echo "Backup completed: backup_${TIMESTAMP}.tar.gz"
```

B.2 disk.sh (conceptual structure)

```
#!/bin/bash
# Disk usage monitoring script
echo "Filesystem usage report - $(date)"
df -h
```

These listings represent the general structure and intent of the scripts created in Steps 6 and 7; the executed output of the actual scripts is captured in Figure 4.

20. Glossary of Terms

- AMI (Amazon Machine Image): a template used to launch an EC2 instance, containing the OS and pre-installed software.
- EC2 (Elastic Compute Cloud): AWS's service for provisioning resizable virtual servers.
- SSH (Secure Shell): a cryptographic protocol for securely accessing a remote machine's command line.
- UFW (Uncomplicated Firewall): a user-friendly front end for managing iptables-based firewall rules on Ubuntu.
- Systemd: the init system and service manager used by modern Ubuntu to start, stop, and monitor background services.
- UID (User ID): a unique numeric identifier assigned to every Linux user account.
- GID (Group ID): a unique numeric identifier assigned to every Linux group.
- Shell script: a text file of sequential shell commands, executed by an interpreter such as bash.
- Security Group: an AWS-level virtual firewall controlling inbound and outbound traffic to an instance.
- Document root: the directory from which a web server serves files (e.g. /var/www/html for Apache).
- VPC (Virtual Private Cloud): an isolated virtual network within AWS in which resources such as EC2 instances are launched.
- Internet Gateway: a VPC component that allows communication between resources in the VPC and the public internet.
- Key pair (.pem): the public/private cryptographic key pair used for SSH authentication to an EC2 instance.
- Daemon: a background process (service) that runs without direct user interaction, such as apache2.
- Package manager: software (e.g. APT) responsible for installing, updating, configuring, and removing software packages consistently.
- Root filesystem: the top-level (/) filesystem containing the core operating system files.
- Home directory: a per-user directory (e.g. /home/adb) where personal files and shell configuration are stored.

- Cron: a time-based job scheduler in Unix-like systems, commonly used to automate recurring script execution.
- Least privilege: a security principle of granting only the minimum access necessary to perform a task.

19a. Appendix C: AWS Console Launch Walkthrough

For reference, the following narrative summarizes the AWS Management Console steps that precede the command-line work shown in this report — used to originally provision the EC2 instance connected to in Step 1.

18. Sign in to the AWS Management Console and navigate to the EC2 service dashboard.
19. Click 'Launch Instance' and provide a descriptive name for the instance.
20. Select an Amazon Machine Image (AMI) — Ubuntu Server 26.04 LTS was chosen.
21. Select an instance type — a Free-Tier-eligible micro instance type was chosen to avoid incurring charges.
22. Create or select an existing key pair (.pem), which is downloaded once and must be stored securely, since AWS does not retain a copy.
23. Configure network settings: select or create a VPC and subnet, and ensure 'Auto-assign public IP' is enabled so the instance is reachable from the internet.
24. Configure the security group to allow inbound SSH (port 22) and later HTTP (port 80) traffic.
25. Configure storage — the default 8GB gp3 root volume was retained, which was sufficient for this project's scope.
26. Review all settings and click 'Launch Instance'.
27. Wait for the instance state to reach 'Running' and note down its public IPv4 address, which is then used for the SSH connection shown in Figure 1.

With the instance running and its public IP address known, administration proceeds entirely over SSH as documented from Step 1 onward in Section 9.

20a. Administrative Best Practices Checklist

As a practical takeaway, the following checklist summarizes best practices demonstrated or referenced throughout this project, useful as a quick pre-deployment review for similar Linux server setups.

Before Going Live

- Update the package index and apply available upgrades
- Confirm key-based SSH authentication is working before disabling any fallback access
- Allow SSH in the firewall before enabling it
- Verify the target service (e.g. Apache) is enabled to survive a reboot, not just started

Ongoing Operations

- Regularly review systemctl status output for unexpected restarts or failures
- Periodically audit /etc/passwd and group memberships for accounts that should no longer have access
- Run disk-usage checks routinely rather than only when storage issues are already visible

- Keep firewall rules minimal — only open ports that are actually required

Documentation and Accountability

- Use individually named accounts rather than shared logins
- Record configuration changes and the reasoning behind them, as done throughout this report
- Maintain an up-to-date command and script reference for the team, similar to Appendix A and B of this document

21. References

- Ubuntu Server Documentation — <https://ubuntu.com/server/docs>
- Apache HTTP Server Documentation — <https://httpd.apache.org/docs/2.4/>
- AWS EC2 User Guide — <https://docs.aws.amazon.com/ec2/>
- UFW — Uncomplicated Firewall Documentation — <https://help.ubuntu.com/community/UFW>
- Linux man pages — `adduser(8)`, `usermod(8)`, `groupadd(8)`, `systemctl(1)`
- AWS Well-Architected Framework — Security Pillar — <https://aws.amazon.com/architecture/well-architected/>